

Juego de plataformas con editor de mapas

Estudiante:	Adalberto Josué Mejías Bonilla
Carnet:	2026100990
Curso:	Introducción a la Programación
Profesor:	Jeff Peralta
Institución:	Tecnológico de Costa Rica
Fecha:	10/05/2025

Enlace del repositorio de github:

<https://github.com/Adal-000/Proyecto-Juego-2D>

1. Objetivo

El minijuego tiene un objetivo sencillo: el usuario controla un personaje tipo slime y debe llegar a la meta establecida en el menor tiempo posible. Durante el recorrido se presentan varias adversidades, como un slime rojo que patrulla zonas específicas, una bola de fuego que sube y baja bloqueando el paso en ciertas partes, y un lanzador de proyectiles que permanece fijo pero complica el avance del jugador.

Aunque el jugador no consiga llegar a la meta en el menor tiempo, puede mantener una puntuación alta recogiendo monedas. Estas monedas funcionan como objetos consumibles que aumentan el puntaje. También se incluyen bloques y escaleras; los bloques permiten desplazarse por el mapa y las escaleras permiten subir o bajar en zonas específicas.

2. Solución

La solución se basa en un juego de plataformas 2D donde el usuario controla al personaje usando el teclado. La barra espaciadora permite saltar, la flecha izquierda mueve al personaje hacia la izquierda y la flecha derecha lo mueve hacia la derecha. Además, las escaleras permiten que el jugador suba o baje cuando se encuentra cerca de ellas.

Los enemigos tienen movimientos definidos para que se mantengan dentro del mapa. El slime rojo se mueve de derecha a izquierda y luego de izquierda a derecha, repitiendo este comportamiento en forma de bucle. La bola de fuego también trabaja en bucle, moviéndose de arriba hacia abajo y luego de abajo hacia arriba. El lanzador de proyectiles permanece fijo y dispara hacia la dirección indicada; cuando el proyectil desaparece, vuelve a lanzar otro.

Para evitar que el mapa sea imposible de jugar, se utiliza una validación basada en el algoritmo BFS. Esta validación revisa si existe al menos un camino posible desde el inicio hasta la meta antes de permitir jugar el mapa.

3. Generación y validación del mapa

La validación del mapa se realiza con el algoritmo BFS, el cual revisa los caminos posibles desde el punto de inicio hasta la meta. Se puede entender como una búsqueda que inicia en una posición y va revisando los lugares cercanos que el jugador podría alcanzar, hasta comprobar si existe una ruta válida.

El sistema va agregando los espacios a los que el jugador puede acceder y los revisa uno por uno. Si se encuentra una ruta hasta la meta, el mapa se toma como jugable. En caso contrario, el mapa no debería aceptarse como válido.

La variedad de mapas se da principalmente por los mapas que los usuarios pueden crear con el editor. Además, el juego cuenta con un mapa genérico para poder hacer pruebas y jugar sin depender siempre de un mapa creado por el usuario.

4. Funciones principales

Las funciones del juego trabajan en conjunto, por lo que es difícil separar completamente unas de otras. Aun así, las más importantes para el funcionamiento general son las siguientes:

- Movimiento del jugador: permite que el personaje se desplace, salte e interactúe con escaleras dentro del mapa.

- Movimiento de enemigos: controla el desplazamiento en bucle del slime rojo, la bola de fuego y los proyectiles del lanzador.
- Validación del mapa: revisa que el mapa tenga inicio, meta y que exista un camino posible para llegar a la meta.
- Colisiones e interacciones: detecta cuando el jugador toca enemigos, trampas, monedas, bloques, escaleras o la meta.

5. Problemas encontrados y solución aplicada

Durante el desarrollo se presentaron varios problemas. Algunos fueron de interfaz, otros de movimiento y otros relacionados con las colisiones del juego. Los principales fueron:

- Interfaz movable: se bloqueó la opción de agrandar o disminuir la ventana para evitar que los elementos se desordenaran.
- Interfaz diferente según el equipo: se utilizó un comando para detectar el tamaño de la pantalla y se acomodaron los elementos con columnas en lugar de usar solamente coordenadas fijas.
- Diccionario incompleto: se fue completando poco a poco conforme se agregaban más elementos y utilidades al juego.
- Botones sin función: se investigaron comandos para abrir nuevas ventanas y se aplicaron dentro de funciones específicas.
- Enemigos saliéndose de los límites: se definió un movimiento en bucle para que no traspasaran las zonas establecidas.
- Monedas que no desaparecían: se corrigieron partes de las hitboxes y de las interacciones entre el jugador y los objetos del mapa.

6. Atribuciones del proyecto

El código del proyecto fue desarrollado de forma original para este juego de plataformas. No se usó código completo copiado de repositorios públicos ni de sitios externos. Las fuentes consultadas fueron usadas como apoyo para comprender herramientas, módulos y conceptos generales.

El proyecto utiliza módulos incluidos en la distribución estándar de Python, principalmente tkinter para la interfaz gráfica, json para archivos y datos, os y sys para rutas e imports, time para el tiempo de juego, copy para copias de mapas, collections para el uso de deque en BFS, y otros módulos estándar relacionados con sonido y manejo de archivos.

También se consultaron conceptos generales como colisiones tipo AABB, búsqueda BFS y generación de audio sintético. Estos conceptos se usaron como referencia teórica para entender cómo aplicarlos dentro del juego.

7. Conclusión

Este proyecto presentó varias dificultades, pero permitió aprender bastante durante el proceso. Fue necesario investigar comandos, probar soluciones y corregir errores que aparecieron en la interfaz, en el movimiento de los enemigos, en las colisiones y en la validación del mapa.

Uno de los aprendizajes más importantes fue entender que un juego no depende solo de mover un personaje, sino de muchas partes conectadas entre sí. La validación del mapa con BFS también permitió conocer un algoritmo reconocido y aplicarlo a una situación concreta. En general, el proyecto ayudó a ganar más experiencia programando y resolviendo problemas de forma ordenada.

8. Referencias

- Microsoft. (s. f.). Visual Studio Code. <https://code.visualstudio.com/>
- OpenAI. (2026). ChatGPT [Modelo de lenguaje grande]. <https://chat.openai.com/>
- Python Software Foundation. (s. f.). collections — Container datatypes. Python documentation. <https://docs.python.org/3/library/collections.html>
- Python Software Foundation. (s. f.). copy — Shallow and deep copy operations. Python documentation. <https://docs.python.org/3/library/copy.html>
- Python Software Foundation. (s. f.). importlib — The implementation of import. Python documentation. <https://docs.python.org/3/library/importlib.html>
- Python Software Foundation. (s. f.). json — JSON encoder and decoder. Python documentation. <https://docs.python.org/3/library/json.html>
- Python Software Foundation. (s. f.). math — Mathematical functions. Python documentation. <https://docs.python.org/3/library/math.html>
- Python Software Foundation. (s. f.). os — Miscellaneous operating system interfaces. Python documentation. <https://docs.python.org/3/library/os.html>
- Python Software Foundation. (s. f.). Python. <https://www.python.org/>
- Python Software Foundation. (s. f.). struct — Interpret bytes as packed binary data. Python documentation. <https://docs.python.org/3/library/struct.html>
- Python Software Foundation. (s. f.). sys — System-specific parameters and functions. Python documentation. <https://docs.python.org/3/library/sys.html>
- Python Software Foundation. (s. f.). time — Time access and conversions. Python documentation. <https://docs.python.org/3/library/time.html>
- Python Software Foundation. (s. f.). tkinter — Python interface to Tcl/Tk. Python documentation. <https://docs.python.org/3/library/tkinter.html>
- Python Software Foundation. (s. f.). wave — Read and write WAV files. Python documentation. <https://docs.python.org/3/library/wave.html>
- Python Software Foundation. (s. f.). winsound — Sound-playing interface for Windows. Python documentation. <https://docs.python.org/3/library/winsound.html>
- Wikipedia contributors. (s. f.). Additive synthesis. En Wikipedia. Recuperado el 10 de mayo de 2025, de https://en.wikipedia.org/wiki/Additive_synthesis
- Wikipedia contributors. (s. f.). Breadth-first search. En Wikipedia. Recuperado el 10 de mayo de 2025, de https://en.wikipedia.org/wiki/Breadth-first_search
- Wikipedia contributors. (s. f.). Collision detection. En Wikipedia. Recuperado el 10 de mayo de 2025, de https://en.wikipedia.org/wiki/Collision_detection#Axis-aligned_bounding_boxes